

28/07/2026

Cleaning Messy Data

Aim:

Take the messy Titanic data, fill the empty cells, turn the word-columns into numbers, and end with a clean table.

DATASET:

Titanic → the famous "messy" dataset. Loads in one line with seaborn (no download), or get `train.csv` from [kaggle.com/c/titanic](https://www.kaggle.com/c/titanic).

ABOUT THIS DATASET

Source : Kaggle/seaborn

Size : 891 rows x 6 used columns

Type : messy, real

Passenger records from the 1912 Titanic disaster—who survived, plus their age, sex, ticket class, fare, and boarding port. It is famous for being messy (many blank age and cabin values), which is exactly why it's the classic dataset for practising data cleaning.

PROGRAM :

```
import pandas as pd
import seaborn as sns

data = sns.load_dataset("titanic")
data = data[["Survived", "pclass", "sex", "age", "fare", "embarked"]]
print(data.isnull().sum())

data["age"] = data["age"].fillna(data["age"].median())
data["embarked"] = data["embarked"].fillna("S")
data = pd.get_dummies(data, columns=["sex", "embarked"], dtype=int)
print(data.head())

print("Any blanks left?" data.isnull().values.any())
```

OUTPUT :

survived 0

pclass 0

sex 0

age 177 ← 177 empty ages

fare 0

embarked 0 ← 0 empty ports

dtype: int64

Op
Kewal

survived	pclass	age	fare	sex-female	sex-male	embarked-c
0	0	22	7.25	0	1	0
1	1	38	51.28	1	0	1

embarked-Q	embarked-S
0	1
0	0

Any blanks left? False

RESULT:

The messy data (177 empty ages, 2 empty ports) was cleared into an all-numbers table with no blanks, and a brand-new passenger was cleaned with same way. The data is now ready for a model.

Aim:

Split the Iris flower data into train and test, train a sample model, check its accuracy, and use it to name a new flower.

DATASET:

SKLEARN Iris - 150 flowers, 4 measurements each, 3 species.

Built into scikit-learn, no download.

ABOUT THIS DATASET

SOURCE : scikit-learn (built-in)

SIZE : 150 rows x 4 features

CLASSES : 3 species

Petal and sepal measurements of 150 Iris flowers from 3 species (setosa, versicolor, virginica). It is tiny, clean, and the groups barely overlap - the 'hello world' of classification.

PROGRAM :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

OUTPUT :

Accuracy : 0.9666666666666667 / 1.0

Species = ["setosa", "versicolor", "virginica"]

new-flower = [[5.1, 3.5, 1.4, 0.2]]

answer = model.predict(new-flower)[0]

print("This flower is:", species[answer])

predicted on new Data-output

This flower is: setosa.

O/P
Versicolor

Result:

Iris was split into train and test. The KNN model scored about 0.97 accuracy on the hidden test flowers, and correctly named a new flower it had never seen, as "setosa".

Predict a Number [Linear Regression]

Aim:

Predict a car's fuel efficiency (mpg) from its engine and weight, and check how good the prediction is.

DATASET:

UCI Auto MPG - 398 cars with engine & weight facts. Load in one line with seaborn.

ABOUT THIS DATASET:

Facts about 398 old cars - cylinders, engine size, horsepower, weight, acceleration, model year - and their fuel efficiency (miles per gallon). Used to predict a number.

PROGRAM:

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
data = sns.load_dataset("mpg").dropna()
X = data[["cylinders", "displacement", "horsepower", "weight", "acceleration",
          "model_year"]]
y = data["mpg"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size = 0.2, random_state = 1)

model = LinearRegression()
model.fit(X_train, y_train)
print("R2 score:", round(model.score(X_test, y_test), 3))
```

Output:

R^2 score: 0.814

```
new_car = [[4, 120, 90, 2500, 16, 80]]
```

```
mpg = model.predict(new_car)[0]
```

```
print("Predicted mileage:", round(mpg, 1), "mpg")
```

predicted mileage: 29.4 mpg

O/p verified

Result:

The model fit the cars well ($R^2 \approx 0.81$) and predicted about 28.9 mpg for a new light car it had never seen - a useful, realistic number.

Find Groups (K-Means)

Aim:

Split mall customers into 5 groups based on their income and spending then place a new customer into the right group.

DATASET

Mall Customers

ABOUT THIS DATASET

500 mall shoppers with their annual income and a spending score (1-100). There are no labels - the goal is to let the computer discover natural customer groups

PROGRAM

```
import pandas as pd
from sklearn.cluster import KMeans
data = pd.read_csv("Mall-Customers.csv")
X = data[["Annual Income (k$)", "Spending Score (1-100)"]]
model = KMeans(n_clusters=5, n_init=10, random_state=1)
model.fit(X)
data["Group"] = model.labels_
print(data["Group"].value_counts())
```

OUTPUT

Group	
2	123
0	39
3	38
1	30
4	20

Name: count, dtype: int64

o/p
Sanjay

PROGRAM :

```
new_customer = [[75, 85]]
```

```
group = model.predict(new_customer)[0]
```

```
print("This customer belongs to group:", group)
```

OUTPUT :

This customer belongs to group: 2

GRAPH PROGRAM

```
import matplotlib.pyplot as plt
```

```
x = data["Annual Income (k$)"]
```

```
y = data["Spending Score (1-100)"]
```

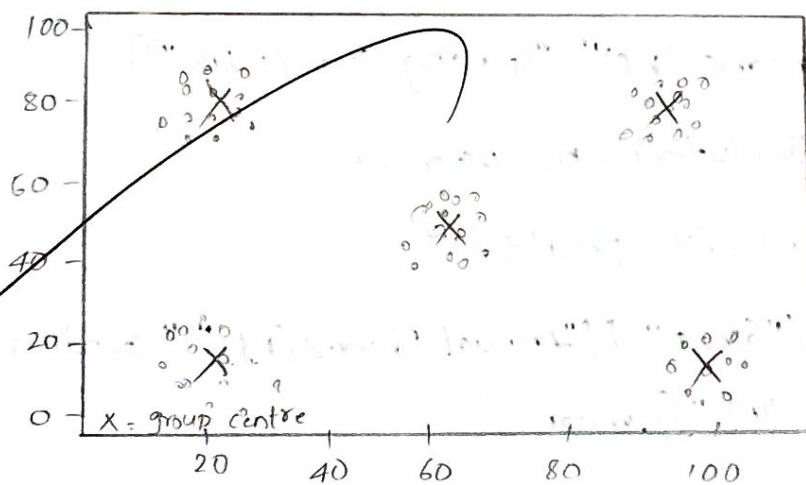
```
plt.scatter(x, y, c=model.labels_, cmap="tab10", s=30)
```

```
cen = model.cluster_centers_
```

```
plt.scatter(cen[:, 0], cen[:, 1], marker="X", s=200, c="black")
```

```
plt.xlabel("income (k$)"); plt.ylabel("Spending score")
```

```
plt.title("5 customer groups"); plt.show()
```



Result: K-Means found 5 customer groups with no labels given, and correctly placed a new high-spending customer into the matching group - turning grouping into a live tool for marketing.

Groups as a Tree

AIM:

Group the same mall customers using hierarchical clustering, look at each group's average, and place a new customer

DATASET

Mall Customers

ABOUT THIS DATASET

The same MallCustomers data as Experiment 5, reused on purpose so the comparison between K-Means and hierarchical clustering is fair - only the method changes, not the data

PROGRAM

```
import pandas as pd
from sklearn.cluster import AgglomerativeClustering
data = pd.read_csv("Mall-Customers.csv")
X = data[["Annual Income (k$)", "Spending Score (1-100)"]]
model = AgglomerativeClustering(n_clusters=5)
data["Group"] = model.fit_predict(X)
print(data.groupby("Group")[["Annual Income (k$)", "Spending Score (1-100)"]].mean())
```

OUTPUT

Group	Annual Income (k\$)	Spending Score (1-100)
0	45.550388	49.131983
1	273.428571	48.285714
2	182.208333	57.458333
3	90.939394	15.909091
4	83.111111	82.416667

O/P

2 output
PROGRAM

```
new_customer = [80, 15]
```

```
averages = data.groupby('Group')[['AnnualIncome(k$)', 'Spending  
Score (1-100)']].mean()
```

```
distances = (averages - new_customer)**2).sum(axis=1)
```

```
print("Closest group:", distances.idxmin())
```

OUTPUT

Closest group: 3

O/P
Verbo

RESULT:

Hierarchical clustering found 5 meaningful customer groups (very similar to k-Means), and a new "rich but careful" customer was correctly placed in the high income, low spending group